

SGI — Prompt Général de la Solution

Document maître. Ce prompt décrit l'intégralité de la solution **SGI** (Système de Gestion Immobilière) telle qu'elle a été développée pour **Infinity International Facilities Management UAE**. Il sert de brief unique : donné à un développeur ou à un agent IA, il permet de comprendre, reconstruire ou étendre la plateforme sans rien omettre des invariants de sécurité et des règles métier.

1. Mission & contexte produit

Construis et fais évoluer **SGI**, un SaaS **multi-tenant** de gestion immobilière de bout en bout pour le marché des **Émirats Arabes Unis** (Dubai, Abu Dhabi), à vocation internationale.

- **Domaine couvert** : catalogue de biens · ventes · locations · CRM · contrats · **Golden Visa UAE**.
- **Utilisateurs** : 50+ agents, managers, juristes, comptables d'une même agence, isolés par société.
- **Langues** : **Arabe RTL (primaire)** · Anglais · Français — trilingue de bout en bout.
- **Devise** : **AED (dirham)**, toujours en chiffres latins, stockée en `DECIMAL(15,2)`.
- **Conformité by-design** : **PDPL** (UAE Data Office), **KYC/AML/goAML**, **TDRA**, **RERA/Ejari/DLD/DEWA/ADDC**.

Le produit est une plateforme web (back-office + portail public), mobile (Expo) et API, pensée **Arabe-RTL-first** et **secure-by-default**.

2. Les 3 Lois architecturales — tout PR qui les viole est bloqué

Loi 1 — Multi-Tenant : `company_id` partout

Chaque table métier porte `company_id` `UUID NOT NULL` + index + **RLS PostgreSQL**. Un middleware FastAPI (`TenantMiddleware`) décode le JWT et exécute `SET LOCAL app.current_company_id` à chaque requête. **Jamais de requête sans filtre `company_id`**. Tables exemptes : `companies`, `users`, `audit_logs`, `countries`.

Application réelle (split de rôles, migration 0022) : la RLS n'est active que si la connexion n'est ni superuser ni propriétaire. L'API se connecte via le rôle restreint `sgi_app` (`NOSUPERUSER`, `NOBYPASSRLS`, non-propriétaire). Le worker Celery et les migrations gardent `sgi_user` (privilegié). **En prod : définir `APP_DB_PASSWORD`**, sinon la RLS est inerte.

```
ALTER TABLE {t} ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON {t}
  USING (company_id = current_setting('app.current_company_id')::UUID);
CREATE INDEX idx_{t}_company ON {t}(company_id);
```

Loi 2 — PostGIS : le géospatial vit dans la base

Colonne `location` `GEOMETRY(Point, 4326)` sur `properties`, **index GIST obligatoire**. Distances toujours en `::geography` (mètres), jamais `::geometry` (degrés). Géocodage une seule fois via Google Maps à la création, résultat stocké en base. Requête rayon via `ST_DWithin` + `ST_Distance` ordonnée.

Loi 3 — RTL Arabe : CSS logique uniquement

Interdit dans les composants partagés : `ml-*` `mr-*` `pl-*` `pr-*` `left-*` `right-*` `text-left/right` `border-l/r`. **Obligatoire** : `ms-*` `me-*` `ps-*` `pe-*` `start-*` `end-*` `text-start/end` `border-s/e`. `DirectionProvider` enveloppe le `RootLayout` ; police Noto Sans Arabic en locale AR.

3. Doctrine de sécurité transversale `@core/security`

7 principes non négociables : Zero Trust · Defense in depth · Secure-by-default · **Fail-secure** (doute/erreur → refuser) · OODA continue · **humain dans la boucle** (actions destructrices déclenchées par un humain) · Privacy by design (PDPL).

3 invariants verrouillés, jamais désactivables (= les 3 Lois, déjà en place) : **Identité** (JWT + Infinity ID sur tout endpoint) · **Isolation tenant** (RLS + `TenantMiddleware`) · **Audit** (`audit_logs` + `AuditMiddleware`).

Contrôles tunables (MFA renforcée, rate-limit strict, honeytokens, UEBA, step-up) : désactivables seulement via workflow d'approbation à 2 + trace `audit_logs` + $TTL \leq 72$ h + alerte critique.

Règles d'or sur chaque module : scoping `company_id` issu du **contexte** (jamais d'une valeur client) · aucun secret en dur (`os.getenv()` / Vault) · chiffrer les champs sensibles (Emirates ID, IBAN, PDC) · émettre les events d'audit aux points sensibles · **test cross-tenant obligatoire** (Red-Team Loi 1 : toute réponse $\neq 404$ = no-go).

Statut : le SDK `@core/security`, les manifestes par module, Vault, OPA/Cedar et Keycloak sont une **doctrine cible** (pas encore implémentés). Seuls les 3 invariants ci-dessus sont la réalité.

4. Stack technique

Couche	Technologie
Frontend	Next.js 15.3 (App Router · Turbopack) · React 19 · TypeScript 5 strict
UI	shadcn/ui (RTL) · Radix UI · Tailwind CSS v4
State	TanStack Query v5 · Zustand v4 · Zod v3
i18n	i18next · AR/EN/FR · Noto Sans Arabic · Geist
Mobile	Expo 51 · React Native 0.74 · Expo Router · NativeWind · react-native-maps · MMKV
Backend	FastAPI 0.136 · Python 3.13 · Pydantic v2 · Uvicorn + uvloop
ORM	SQLAlchemy 2 async · GeoAlchemy2 · Alembic
Tâches	Celery + Beat · 3 files : notifications, exports, reminders
DB	PostgreSQL 17 + PostGIS 3.5 · RLS · index GIST
Cache	Valkey 8 (fork Redis OSS) · sessions · broker Celery
Recherche	Meilisearch (AR/EN/FR · tolérant aux fautes · self-hosted)
Stockage	MinIO (S3-compatible · médias · contrats · documents)
Infra	Docker Compose · Nginx 1.27 · Certbot · Turborepo + pnpm
Observabilité	Prometheus · Grafana · Loki + Promtail · Sentry
Tests	Vitest · pytest-asyncio · Playwright

5. Architecture du monorepo

```
apps/  
  api/      FastAPI · uv · tests co-localisés app/routers/{module}/test_{module}.py  
  web/      Back-office Next.js (port 5001 local · 3000 Docker)  
  portal/   Portail public Next.js (port 3001)  
  mobile/   Expo / React Native (lockfile npm propre)  
packages/  
  shared-types/  Types TS partagés web/portal  
  i18n/          Bundles de traduction AR/EN/FR  
infra/          nginx · certbot · monitoring (prometheus/grafana/loki)
```

Wiring API : entrée `apps/api/app/main.py` (le lifespan démarre le pool DB et le navigateur Playwright pour le scraping). Ordre middleware (dernier ajouté = premier exécuté) : `CORS` → `TenantMiddleware` → `AuditMiddleware` → `GZip` — **ne pas réordonner**. Tous les routeurs sont montés sous `/api/v1`. Migrations Alembic dans `apps/api/migrations/versions/` (0001 → 0067, head `0067_studio_orchestrator_jobs`).

Pattern uniforme de module (apps/api/app/routers/{module}/) : router.py (endpoints + auth + pré-handlers tenant) · schemas.py (Pydantic v2) · service.py (logique, toujours filtrée par company_id) · models.py · test_{module}.py · CLAUDE.md (règles métier). Sous-module IA optionnel : ai_router.py / ai_service.py / ai_schemas.py, monté sous /{module}/ai/..., **404 (jamais 403)** sur cross-tenant pour éviter l'oracle d'existence (BOLA).

6. Modules fonctionnels (≈ 51 routeurs)

Cœur immobilier : properties, realestate_core, buildings, units, owners, tenants, vendors, technicians (pattern party-role + hiérarchie physique bâtiment/étage/unité).

Commercial & transactions : crm, clients, acquisitions, sales, leasing, rentals, developers, marketing, sources, public_site, social, scenarios.

Contrats & conformité : contracts (+ signature de renouvellement), golden_visa, pdc (chèques post-datés, machine à états UAE), documents.

Finance : finance, payments, accounting (**grand livre en partie double** — Σ débits == Σ crédits, Decimal jamais float, company_id dénormalisé sur les lignes), bank (import relevés

- réconciliation), owner_statements, owner_portal.

Opérations & relation : maintenance, inspections, workflows, comms, notifications, agenda, inbox (omnicanal + webhook WhatsApp + WS temps réel), ticketing (SLA + escalade), telephony (centre de contact Asterisk WebRTC + enregistrement PDPL), copilot (assist agent), partner, client_portal, tenant_portal.

Plateforme & sécurité : auth, iam (RBAC hiérarchique, arbre de ressources + héritage), admin (console agrégateur : users / audit / alerts / backups / infra / prometheus + **Studio de Modules** : usine à modules in-app, gouvernance 4-yeux, codegen via worker isolé worker-studio, **jamais d'auto-merge** — ouvre une PR), search (Meilisearch), reporting, ai_services, scraping, honeytokens (leurres, endpoint public neutre 404), self_defense (events → audit_logs), presence (heartbeat ; active?advanced=1 réservé admin).

Infinity ID / UAE Infinity PASS : IdP interne (niveaux d'assurance L0-L3, step-up sur actions sensibles, signature qualifiée maison) inspiré d'UAE PASS mais **non fédéré** au gouvernement.

7. Règles métier clés

CRM — séquence de relance (max 4 tentatives sur 7 jours)

J+1 appel · J+2 WhatsApp (template Meta) · J+4 email personnalisé · J+7 push + WhatsApp. Après la séquence : status='lost', reason='non_respondent' automatique.

Lead scoring (0–100)

Budget ≥ 2 M AED (seuil Golden Visa) +25 · ≥ 500 k +15 · nationalité éligible Golden Visa +20 · type de bien précisé +15 · taux de réponse $\times 20$ · 2+ canaux +10 · contact < 7 j +10.

Pipeline (seules transitions valides)

new → contacted → qualified → proposal_sent ; proposal_sent → visit_planned | negotiation | lost ; visit_planned → visit_done | lost ; visit_done → negotiation | proposal_sent | lost ; negotiation → won | lost . won et lost terminaux.

Automatisations post- won

Vente ≥ 2 M AED → workflow **Golden Visa** · location commerciale → offre de création de société · toute location → alerte de renouvellement à J-120.

Golden Visa

Éligibilité : bien ≥ 2 M AED signé. Documents : passeport · DLD · GDRFA · assurance · photo biométrique. Alertes à J-90 et J-30 avant expiration du visa.

8. Conventions de développement

- **Base** : UUID v4 (gen_random_uuid()), created_at / updated_at partout, **soft delete** (deleted_at, jamais de DELETE physique), montants DECIMAL(15,2) AED, colonnes multilingues *_ar / *_en / *_fr.
- **Backend** : type hints partout (mypy strict), Pydantic v2 pour toutes les I/O, réponse standard {success, data, meta} ou {success, error{code,message}}, endpoints toujours async, Celery pour tout traitement > 500 ms, gestionnaire de paquets uv.
- **Frontend** : strict: true (pas de any), Server Components par défaut ("use client" seulement si nécessaire), devise via Intl.NumberFormat("en-AE", {currency:"AED"}).
- **Commits** : type(module): description courte en français (feat·fix·test·docs·chore·refactor·perf·ci).
- **TDD** : red → code minimal → green → refactor → commit. Couverture ≥ 80 % sur la logique métier (PR bloquée sinon). Deux couches de tests co-localisées : helpers purs (sans DB) + intégration endpoint (Postgres réel, dans le conteneur).

9. Processus de dev & gate de fusion (obligatoire)

Règle d'or : jamais de dev direct sur main — main reste déployable en permanence.

1. **Branche dédiée** type/module-description, build & tests dans un **git worktree isolé** sur origin/main. Respect des 3 Lois dès le dev. Pré-push : uv run ruff format + ruff check + mypy (la CI échoue sinon).

2. **2 agents séquentiels : Agent Test Fonctionnel** (pytest en conteneur, vitest, `tsc --noEmit`, `ruff`, couverture $\geq 80\%$, exercice réel des endpoints) → puis **Agent Audit Sécurité** (adversarial : 3 Lois + OWASP + zones sensibles : MFA, BOLA, authz WebSocket, rôle `sgi_app`, secrets).
3. **Locataire Red-Team automatisé** : provisionne une 2^e société éphémère et rejoue chaque endpoint du diff en cross-tenant + cross-utilisateur. **Toute réponse $\neq 404$ / vide = no-go bloquant.**
4. **Carte de Fusion** (scorecard go/no-go par dimension) postée en commentaire de PR ; merge **scellé par le GO explicite de l'utilisateur**. Aucun agent ne merge de lui-même.
5. **Fusion** sans `--delete-branch`, suppression de la branche distante séparément, resync de `main`.

Orchestré par le skill `/centre-de-commande` : architecte → dev →  RADAR (test) →  CHASSEUR (audit Red-Team) →  DÔME DE FER (intégration supervisée) → Carte de Fusion → worktree/PR → attente du « GO #PR ».

10. Anti-patterns — toujours refuser

- Requête sans filtre `company_id` (violation Loi 1).
 - `DELETE` physique au lieu du soft delete.
 - CSS physique (`ml-4 text-left`) cassant le RTL.
 - `lat / lng` en float au lieu de PostGIS.
 - Requête N+1 dans une boucle.
 - Secret en dur (`api_key = "sk-..."`) — utiliser `os.getenv()`.
 - `"use client"` sur des pages data-heavy (gaspille le RSC).
 - Volumes Docker anonymes · cache Valkey sans TTL · table métier sans `company_id`.
-

11. Commandes essentielles

```
# Infrastructure
make up / make down / make logs s=api / make healthcheck
make monitoring / make monitoring-down      # stack observabilité opt-in

# Base de données
make migrate          # alembic upgrade head (rôle privilégié sgi_user)
make seed             # données de test Dubaï
make backup           # pg_dump compressé → ./backups/

# Dev frontend (Turborepo + pnpm)
pnpm dev              # toutes les apps front
pnpm dev --filter=sgi-web    # back-office (5001 local · 3000 Docker)
pnpm dev --filter=@sgi/portal # portail public (3001)
pnpm typecheck         # tsc --noEmit sur toutes les apps

# Tests
make test              # suite complète
docker compose exec api uv run pytest app/routers/crm/test_crm.py # un module backend
pnpm vitest run · pnpm playwright test

# Qualité & scaling
make lint              # ESLint + Ruff + Prettier
make scale n=5         # FastAPI à N répliques
```

Document généré comme prompt général synthétisant l'intégralité de la solution SGI développée pour Infinity International Facilities Management UAE.